

Esercizio 4 — Wireshark: traffico HTTP e HTTPS

Autore: Francesco Terracciano · **Data:** 7 settembre 2026 · **Ambiente:** Kali Linux (VM ARM64 su UTM) · **Interfaccia:** eth0 (192.168.64.9)

Obiettivo del laboratorio: catturare con tcpdump del traffico HTTP e HTTPS e analizzarlo con Wireshark per dimostrare che l'HTTP viaggia in chiaro (dati leggibili da chi intercetta) mentre l'HTTPS è protetto dalla cifratura TLS. Di seguito le operazioni svolte nell'ordine reale di esecuzione, con gli output raccolti e le risposte alle domande della traccia.

1 Cattura del traffico HTTP con tcpdump

Individuata l'interfaccia di rete con accesso a internet (`eth0` , IP 192.168.64.9), ho avviato la cattura su file con `sudo tcpdump -i eth0 -s 0 -w httpdump.pcap` . L'opzione `-s 0` imposta lo snaplen al valore pieno, `-w` scrive il traffico nel file .pcap. Ho poi generato traffico HTTP verso un endpoint di test, inviando un form con dati fittizi, e ho fermato la cattura con `CTRL+C` (78 pacchetti).

```
(kali@kali)-[~]
$ sudo tcpdump -i eth0 -s 0 -w httpdump.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C78 packets captured
82 packets received by filter
0 packets dropped by kernel
```

Fig. 1 — `tcpdump -i eth0 -s 0 -w httpdump.pcap` : 78 pacchetti catturati su eth0.

2 Analisi della cattura HTTP in Wireshark

Ho aperto `httpdump.pcap` in Wireshark e applicato il filtro `http` . Compaiono la richiesta **POST** (pacchetto 27, dalla workstation verso il server) e la risposta **HTTP/1.1 200 OK** (pacchetto 35).

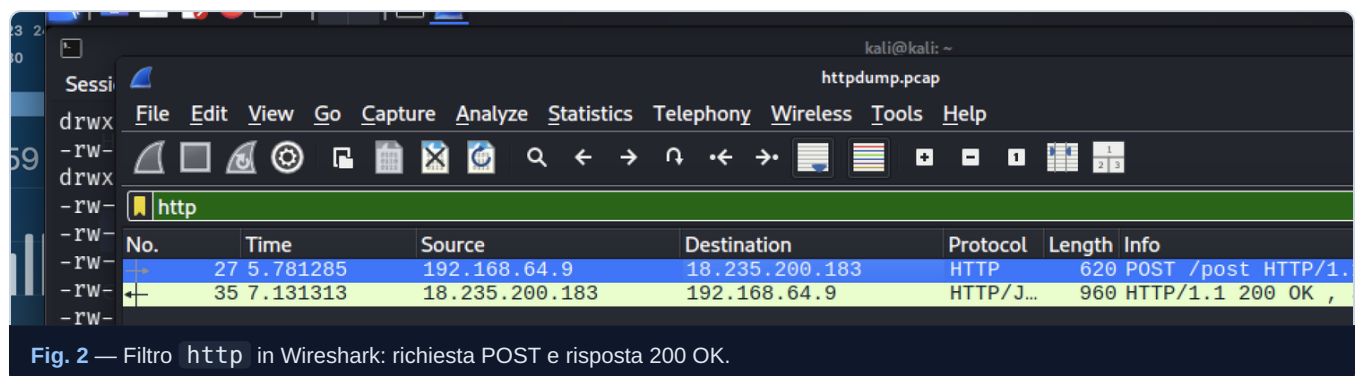
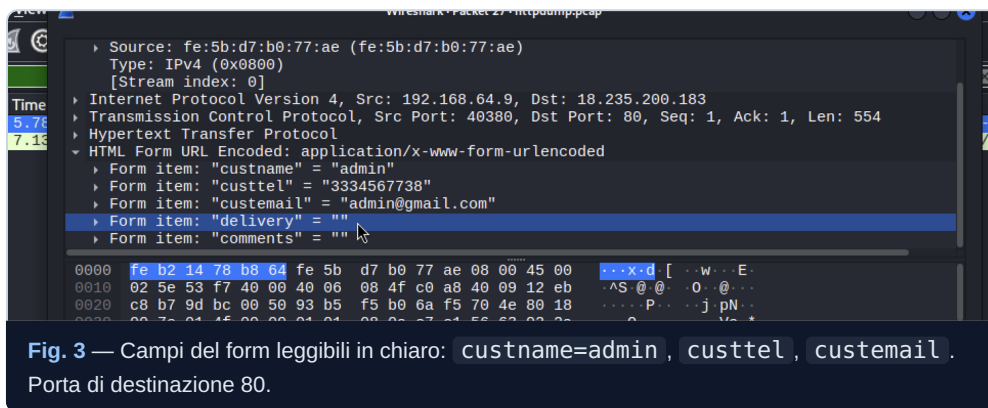


Fig. 2 — Filtro `http` in Wireshark: richiesta POST e risposta 200 OK.

3 Dati del form leggibili in chiaro (HTTP)

Selezionando il messaggio POST ed espandendo la sezione **HTML Form URL Encoded** (`application/x-www-form-urlencoded`), tutti i campi del form risultano **in chiaro** e leggibili da chiunque intercetti il traffico. In questo caso sono esposti nome, telefono ed email di un ipotetico utente.

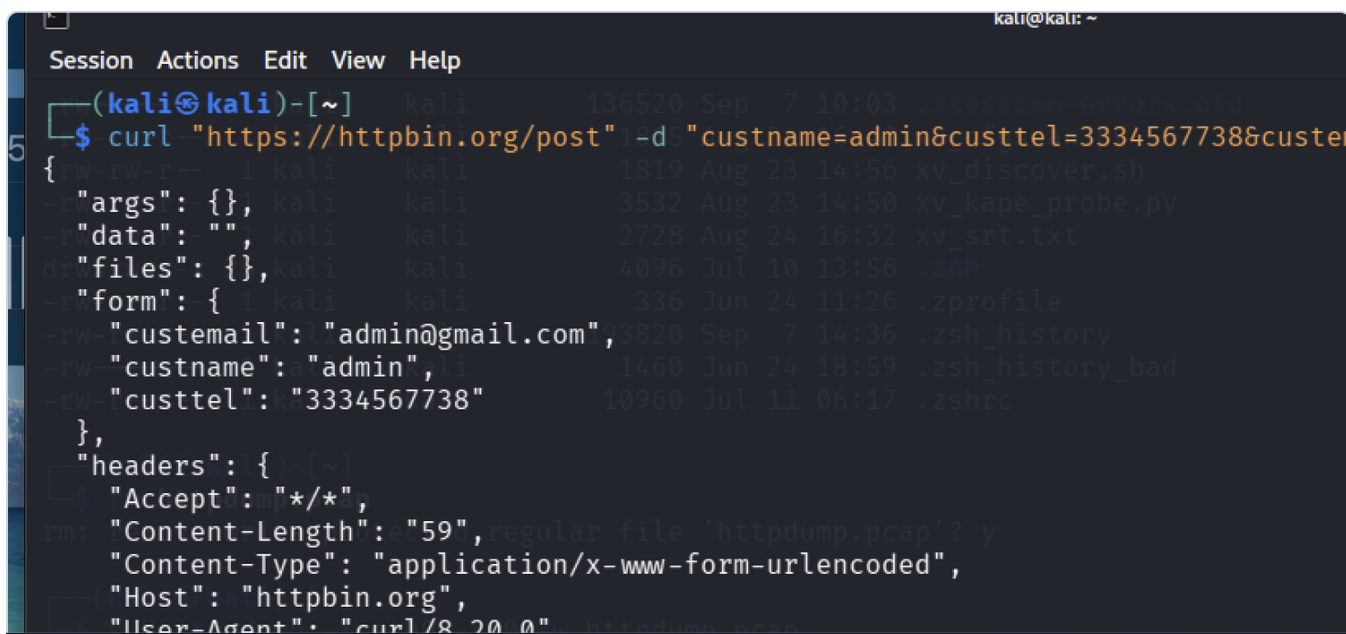


Elenca le interfacce e i loro indirizzi IP.

Dall'output di `ip address`: `lo` (loopback, 127.0.0.1), `eth0` (192.168.64.9/24, usata per la cattura perché ha accesso a internet) ed `eth1` (192.168.50.100/24).

4 Cattura del traffico HTTPS

Ho avviato una nuova cattura con `sudo tcpdump -i eth0 -s 0 -w httpsdump.pcap` e generato traffico verso lo stesso endpoint ma su **HTTPS**, inviando gli stessi dati fittizi. La risposta ricevuta al terminale è leggibile perché già decifrata a destinazione — ma è ciò che vede l'endpoint, non chi intercetta il traffico (vedi passo successivo).



5 Analisi della cattura HTTPS in Wireshark

Aperto `httpsdump.pcap` e filtrato il traffico TLS (porta 443). Si riconosce l'handshake TLS iniziale — **Client Hello** (con SNI del dominio), **Server Hello**, **Certificate**, **Change Cipher Spec** — seguito da una serie di pacchetti **Application Data** e da un **Encrypted Alert** di chiusura.

The image shows a Wireshark packet capture of a TLS handshake and encrypted application data. The filter is set to 'tls'. The packet list shows the following details:

No.	Time	Source	Destination	Protocol	Length	Info
10	0.230740	192.168.64.9	44.195.8.204	TLSv1.2	313	Client Hello (SNI=)
12	0.419826	44.195.8.204	192.168.64.9	TLSv1.2	2962	Server Hello
13	0.419826	44.195.8.204	192.168.64.9	TLSv1.2	1433	Certificate, Serve
18	0.422512	192.168.64.9	44.195.8.204	TLSv1.2	192	Client Key Exchang
20	0.535533	44.195.8.204	192.168.64.9	TLSv1.2	117	Change Cipher Spec
21	0.535533	44.195.8.204	192.168.64.9	TLSv1.2	135	Application Data
23	0.536406	192.168.64.9	44.195.8.204	TLSv1.2	300	Application Data
24	0.536594	192.168.64.9	44.195.8.204	TLSv1.2	130	Application Data

Fig. 5 — Filtro `tls` : handshake TLS (Client/Server Hello, Certificate) e pacchetti Application Data cifrati.

6 Contenuto cifrato e non leggibile (HTTPS)

Selezionando un pacchetto ed espandendo la sezione **Transport Layer Security**, il contenuto applicativo non è più leggibile: al posto dei campi del form c'è solo "Application Data" cifrato. Persino il messaggio di chiusura (pacchetto 31) è un **Encrypted Alert** — anche i controlli TLS sono cifrati.

The image shows the details of packet 31 in Wireshark. The packet is a TLSv1.2 Record Layer: Encrypted Alert. The details pane shows the following information:

- Frame 31: Packet, 97 bytes on wire (776 bits), 97 bytes captured (776 b)
- Ethernet II, Src: fe:5b:d7:b0:77:ae (fe:5b:d7:b0:77:ae), Dst: fe:b2:14:78:b8:64
- Internet Protocol Version 4, Src: 192.168.64.9, Dst: 44.195.8.204
- Transmission Control Protocol, Src Port: 45226, Dst Port: 443, Seq: 2175
- Transport Layer Security
 - [Stream index: 0]
 - TLSv1.2 Record Layer: Encrypted Alert
 - Content Type: Alert (21)
 - Version: TLS 1.2 (0x0303)
 - Length: 26
 - Alert Message: Encrypted Alert

The packet bytes pane shows the hex dump: 0000 fe b2 14 78 b8 64 fe 5b d7 b0 77 ae 08 00 45 00 ... x . d . [. w . . E

Fig. 6 — Dettaglio del pacchetto 31: `TLSv1.2 Record Layer: Encrypted Alert`, contenuto cifrato e non leggibile.

Cosa noti riguardo all'URL del sito web (HTTPS)?

Inizia con `https://` (non `http://`) e il browser mostra l'icona del lucchetto: indicano che la connessione è cifrata tramite TLS. Nella cattura questo si riflette nell'uso della porta 443 e nell'SNI del dominio presente nel Client Hello.

Cosa ha sostituito la sezione HTTP che era nel file di cattura precedente?

La sezione **Transport Layer Security** con i pacchetti **Application Data**. Dove in HTTP c'era "Hypertext Transfer Protocol / HTML Form URL Encoded" con i campi in chiaro, in HTTPS c'è solo il protocollo TLS: handshake iniziale e poi dati applicativi cifrati.

I dati dell'applicazione sono in formato plaintext o leggibile?

No. I pacchetti Application Data sono cifrati e illeggibili: nel dettaglio compare solo "Application Data" e nel dump esadecimale byte casuali, non testo. Anche l'alert di chiusura è cifrato (Encrypted Alert).

Domande di riflessione

1. Quali sono i vantaggi dell'uso di HTTPS invece di HTTP?

HTTPS aggiunge la cifratura TLS, garantendo tre proprietà assenti in HTTP: **riservatezza** (i dati — credenziali, dati personali, contenuti — non sono leggibili da chi intercetta il traffico, come dimostrato: in HTTP i campi del form erano in chiaro, in HTTPS solo Application Data cifrato); **integrità** (i dati non possono essere alterati in transito senza che venga rilevato); **autenticazione** (il certificato del server consente di verificare l'identità del sito). In sintesi protegge da sniffing e attacchi man-in-the-middle.

2. Tutti i siti web che usano HTTPS sono considerati affidabili?

No. HTTPS garantisce che la connessione sia cifrata, non che il sito sia onesto. Gli attori malevoli usano comunemente HTTPS per nascondere le proprie attività: un sito di phishing o che distribuisce malware può avere un certificato valido e mostrare il lucchetto. Il lucchetto indica solo che la comunicazione è protetta, non che il server sia affidabile: occorre sempre valutare reputazione e identità del sito.

Esito. Il confronto tra le due catture dimostra la differenza tra i due protocolli: in HTTP i dati del form (nome, telefono, email) sono stati letti in chiaro direttamente dal pacchetto POST; in HTTPS gli stessi dati sono risultati cifrati e illeggibili, con visibile solo l'handshake TLS. L'unica informazione ricavabile da un intercettatore in HTTPS è l'identità dell'host contattato (SNI/IP), non il contenuto scambiato.

Nota metodologica: l'esercizio è stato svolto su Kali Linux ARM64 (anziché sulla CyberOps Workstation) generando il traffico via `curl` verso `httpbin.org`, poiché i siti di test indicati nella traccia non erano raggiungibili. Il filtro TLS è stato usato al posto di `tcp.port==443` (equivalente). I concetti dimostrati e i risultati sono identici a quelli previsti dalla traccia.